



CS-282 Sistemas de Software Seguro

Professor Cesar Augusto Cavalheiro Marcondes

Exame – DevSecOps com LLM

18 de Junho de 2026

Arthur Rocha e Silva
Gabriel Padilha Leonardo Ferreira
Marcelo Hippolyto de Sandes Peixoto

Engenharia de Computação
Instituto Tecnológico de Aeronáutica – ITA

1. Introdução

Este relatório documenta a aplicação de uma esteira de DevSecOps assistida por LLM sobre o DVPWA (Damn Vulnerable Python Web Application). O fluxo seguiu cinco etapas:

1. scan inicial com as ferramentas Bandit, Semgrep, pip-audit e Gitleaks;
2. triagem dos achados com auxílio de uma LLM;
3. remediação (sugestão de correções) com a LLM;
4. aplicação manual dos patches e commits;
5. segunda esteira para validar as correções.

Escolhemos três vulnerabilidades para o ciclo completo: injeção de SQL, armazenamento fraco de senha (MD5) e falta de hardening do contêiner Redis. Todos os artefatos (relatórios, prompts, saídas da LLM e patches) estão versionados no repositório.

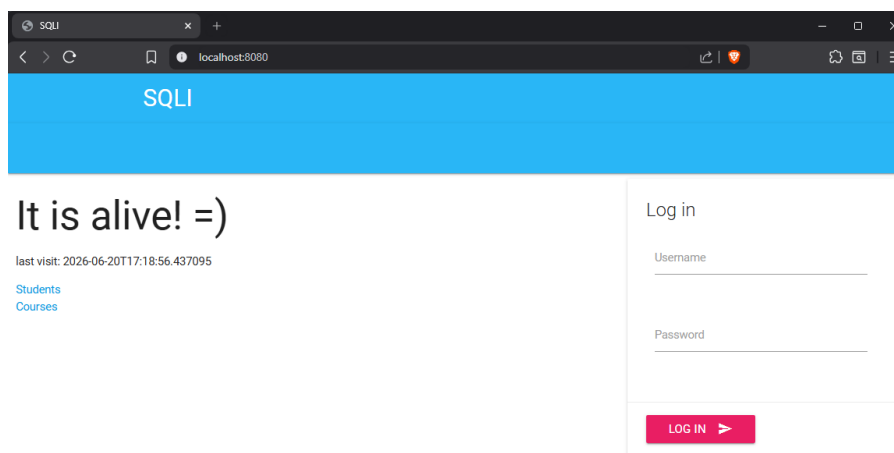


Figura 1: DVPWA em execução localmente via Docker Compose.

2. Primeira Esteira: Diagnóstico

A primeira esteira rodou as quatro ferramentas sobre o código original, antes de qualquer correção, salvando os relatórios em `artifacts/scan-before/`. Em resumo:

- Bandit: B608 (SQL montado por string) e B324 (uso de MD5);
- Semgrep: query SQL formatada, MD5 para senha e falta de hardening no Redis, além de 3 alertas INFO em `materialize.js`;
- pip-audit: nenhuma dependência vulnerável;
- Gitleaks: nenhum segredo detectado.

3. Triagem e Remediação com LLM

Os relatórios e um prompt de triagem foram fornecidos a uma LLM, gerando o arquivo `llm-triage.md`. A LLM agrupou achados duplicados, identificando que a injeção de SQL apareceu três vezes, sendo uma pelo Bandit e duas por regras do Semgrep. Também identificou que o uso de MD5 apareceu duas vezes.

Além disso, a LLM apontou falsos positivos. Os três alertas de format string em `materialize.js` foram classificados como pertencentes a uma biblioteca de terceiros minificada, sem caminho de exploração claro. Com base nessa triagem, a LLM priorizou os achados realmente exploráveis e selecionou três vulnerabilidades para correção.

Em uma segunda interação, a LLM recebeu a triagem e os trechos de código correspondentes, produzindo o arquivo `llm-remediation.md`. Para cada vulnerabilidade, a saída apresentou a causa raiz, um patch mínimo e seguro, um teste de regressão e a forma de confirmar a correção no segundo scan. As sugestões foram revisadas e aplicadas manualmente.

4. Testes

Antes de aplicar as correções, os testes foram escritos e executados contra o código ainda vulnerável. Essa etapa serviu para confirmar que as falhas existiam e que os testes seriam capazes de detectar a correção posteriormente.

Foram usados dois tipos de testes. Os testes funcionais verificam comportamentos esperados que devem continuar funcionando após os patches, como a criação de um aluno gerando um INSERT e a rejeição de uma senha incorreta. Já os testes de regressão reproduzem diretamente cada vulnerabilidade e só passam quando ela é corrigida, como o tratamento do nome como parâmetro, a verificação de senha com `bcrypt` e o endurecimento do serviço Redis.

```
tests\test_functional.py .. [ 40%]
tests\test_regression.py FFF [100%]

===== FAILURES =====
----- test_student_name_is_passed_as_parameter -----

def test_student_name_is_passed_as_parameter():
    cur = FakeCursor()
    payload = "Robert'); DROP TABLE students; --"
    run(Student.create(FakeConn(cur), payload))
    query, params = cur.calls[0]
> assert payload not in query
E       assert "Robert'); D...students; --" not in "INSERT INTO...students; --)"
E
E       "Robert'); DROP TABLE students; --" is contained here:
E         INSERT INTO students (name) VALUES ('Robert'); DROP TABLE students; --)

tests\test_regression.py:13: AssertionError
----- test_password_is_verified_with_bcrypt -----

def test_password_is_verified_with_bcrypt():
> assert make_user(BCRYPT_HASH).check_password(PASSWORD) is True
E       AssertionError: assert False is True
E       + where False = check_password('s3cret')
E       +   where check_password = User(id=1, first_name='Ada', middle_name=None, last_name='Lovelace', username='ada', pwd_hash='2b$12$FvHFL/LLvehgDUhSyssrPe5IEz5032kz7rcbbMiyC8oI5cp.BqASC', is_admin=False).check_password
E       +   where User(id=1, first_name='Ada', middle_name=None, last_name='Lovelace', username='ada', pwd_hash='2b$12$FvHFL/LLvehgDUhSyssrPe5IEz5032kz7rcbbMiyC8oI5cp.BqASC', is_admin=False) = make_user('$2b$12$FvHFL/LLvehgDUhSyssrPe5IEz5032kz7rcbbMiyC8oI5cp.BqASC')

tests\test_regression.py:18: AssertionError
----- test_redis_service_is_hardened -----

def test_redis_service_is_hardened():
    with open("docker-compose.yml") as f:
        compose = yaml.safe_load(f)
        redis = compose["services"]["redis"]
> assert redis.get("read_only") is True
E       AssertionError: assert None is True
E       + where None = <built-in method get of dict object at 0x00000195E9AC8040>('read_only')
E       +   where <built-in method get of dict object at 0x00000195E9AC8040> = {'image': 'redis:alpine'}.get

tests\test_regression.py:25: AssertionError

===== short test summary info =====
FAILED tests/test_regression.py::test_student_name_is_passed_as_parameter - assert "Robert'); D...students; --" not in "INSERT INTO...ud
ents; --)"
FAILED tests/test_regression.py::test_password_is_verified_with_bcrypt - AssertionError: assert False is True
FAILED tests/test_regression.py::test_redis_service_is_hardened - AssertionError: assert None is True
===== 3 failed, 2 passed in 0.36s =====
```

Figura 2: Execução dos testes antes das correções: regressão falhando e funcionais passando.

5. Correções Aplicadas

As correções foram aplicadas na branch `fix/security-remediation`, com commits separados por vulnerabilidade.

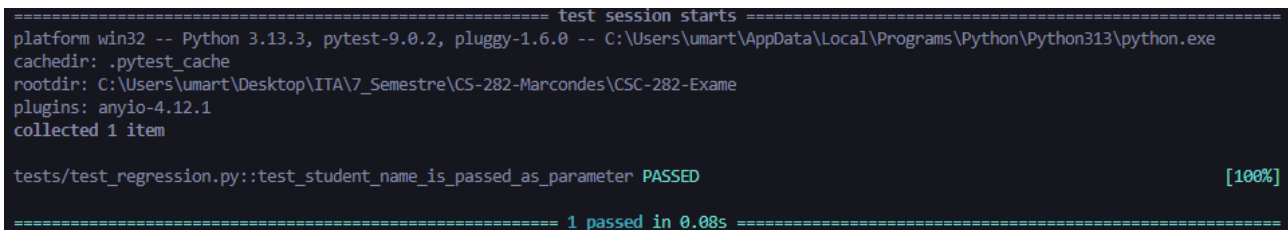
a. Injeção de SQL (`sqli/dao/student.py`)

Causa raiz: a query de INSERT era montada por formatação de string com o nome digitado pelo usuário, e executada já formatada. Isso permite quebrar a string e injetar SQL (CWE-89).

Correção: usar query parametrizada, deixando o driver tratar o valor.

```
# Antes
q = "INSERT INTO students (name) VALUES ('%(name)s')" % {'name': name}
await cur.execute(q)

# Depois
await cur.execute(
    "INSERT INTO students (name) VALUES (%(name)s)", {"name": name})
```



```
===== test session starts =====
platform win32 -- Python 3.13.3, pytest-9.0.2, pluggy-1.6.0 -- C:\Users\umart\AppData\Local\Programs\Python\Python313\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\umart\Desktop\ITA\7_Semestre\CS-282-Marcondes\CSC-282-Exame
plugins: anyio-4.12.1
collected 1 item

tests/test_regression.py::test_student_name_is_passed_as_parameter PASSED [100%]

===== 1 passed in 0.08s =====
```

Figura 3: Teste de regressão da injeção de SQL passando após a correção (parametrização da query em `sqli/dao/student.py`).

b. Armazenamento de senha com MD5 (`sqli/dao/user.py`)

Causa raiz: a verificação de senha usava MD5, que é rápido, sem salt e quebrável, além de fazer uma comparação direta suscetível a timing attack (CWE-327 / CWE-916).

Correção: migrar para `bcrypt` (salt embutido, custo ajustável e comparação em tempo constante). Também foi adicionado um método `hash_password` para centralizar a geração de hash em futuros cadastros. Foram ajustados ainda o `requirements.txt` (`bcrypt`) e as fixtures, que passaram a gerar o hash com `pgcrypto` para que os logins de teste continuem válidos.

```
# Antes
return self.pwd_hash == md5(password.encode('utf-8')).hexdigest()

# Depois
return bcrypt.checkpw(password.encode('utf-8'),
    self.pwd_hash.encode('utf-8'))
```

```
===== test session starts =====
platform win32 -- Python 3.13.3, pytest-9.0.2, pluggy-1.6.0 -- C:\Users\umart\AppData\Local\Programs\Python\Python313\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\umart\Desktop\ITA\7_Semestre\CS-282-Marcondes\CSC-282-Exame
plugins: anyio-4.12.1
collected 1 item

tests/test_regression.py::test_password_is_verified_with_bcrypt PASSED [100%]

===== 1 passed in 0.27s =====
```

Figura 4: Teste de regressão da senha passando após a correção (verificação com bcrypt em `sql/dao/user.py`, substituindo o MD5).

c. Hardening do contêiner Redis (`docker-compose.yml`)

Causa raiz: o serviço redis rodava sem `no-new-privileges` e com `filesystem` raiz gravável, ampliando o impacto de um eventual comprometimento (defesa em profundidade).

Correção: aplicar o princípio do menor privilégio no serviço.

```
# Antes
redis:
  image: redis:alpine

# Depois
redis:
  image: redis:alpine
  read_only: true
  security_opt:
    - no-new-privileges:true
  tmpfs:
    - /tmp
```

```
===== test session starts =====
platform win32 -- Python 3.13.3, pytest-9.0.2, pluggy-1.6.0 -- C:\Users\umart\AppData\Local\Programs\Python\Python313\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\umart\Desktop\ITA\7_Semestre\CS-282-Marcondes\CSC-282-Exame
plugins: anyio-4.12.1
collected 1 item

tests/test_regression.py::test_redis_service_is_hardened PASSED [100%]

===== 1 passed in 0.10s =====
```

Figura 5: Teste de regressão do hardening do Redis passando após a correção (`read_only`, `no-new-privileges` e `tmpfs` no serviço redis do `docker-compose.yml`).

6. Segunda Esteira: Validação

Depois dos patches, rodamos de novo os testes funcionais, os testes de regressão e as quatro ferramentas, salvando os relatórios em `artifacts/scan-after/`. Os cinco testes passam (dois funcionais e três de regressão), confirmando que as correções funcionam e que a aplicação continua íntegra.

```
===== test session starts =====
platform win32 -- Python 3.13.3, pytest-9.0.2, pluggy-1.6.0
rootdir: C:\Users\umart\Desktop\ITA\7_Semestre\CS-282-Marcondes\CSC-282-Exame
plugins: anyio-4.12.1
collected 5 items

tests/test_functional.py .. [ 40%]
tests/test_regression.py ... [100%]

===== 5 passed in 0.57s =====
```

Figura 6: Testes funcionais e de regressão passando após as correções (cinco testes, todos verdes).

A tabela abaixo compara os achados das duas rodadas para as três vulnerabilidades escolhidas e para os itens já classificados na triagem.

Achado	Ferramenta	Antes	Depois	Situação
Injeção de SQL (student.py)	Bandit, Semgrep	3	0	Desapareceu
Senha com MD5 (user.py)	Bandit, Semgrep	2	0	Desapareceu
Hardening do Redis (compose)	Semgrep	2	0	Desapareceu
unsafe-formatstring (materialize.js)	Semgrep	3	3	Permaneceu (falso pos.)
Dependências vulneráveis	pip-audit	0	0	Sem achados
Segredos versionados	Gitleaks	0	0	Sem achados

Tabela 1: Comparação dos achados entre a primeira e a segunda esteira.

As três vulnerabilidades selecionadas deixaram de ser reportadas pelas duas ferramentas de SAST. Os três alertas unsafe-formatstring permaneceram, pois estão na biblioteca de terceiros materialize.js e já eram falsos positivos na triagem. O pip-audit e o Gitleaks seguem sem achados.

A segunda rodada também trouxe achados novos, todos de baixa severidade e fora do código da aplicação:

- Bandit: oito alertas B101 (assert_used) e um B105 (hardcoded_password_string) na pasta tests/. São o uso normal de assert nos testes e o hash bcrypt fixo usado como dado de teste, não credenciais. Aparecem porque o Bandit passou a varrer também a pasta de testes.
- Semgrep: um detect-non-literal-regexp no materialize.js (terceiros) e um detected-bcrypt-hash dentro de artifacts/scan-after/bandit.json, ou seja, a ferramenta acabou lendo o próprio relatório gerado. Ambos são falsos positivos.

Esses ruídos podem ser silenciados com .banditignore e .semgrepignore excluindo as pastas tests/ e artifacts/, sem afetar a análise do código da aplicação.

